

Università degli Studi di Salerno
Corso di Ingegneria del Software

MediBook
ODD
Versione 1.5



Data: 21/12/2025

Progetto: MediBook	Versione: 1.5
Documento: ODD	Data: 21/12/2025

Coordinatore del progetto:

Nome	Matricola
Lelio Crisci	0512119092

Partecipanti:

Nome	Matricola
Samuele Colucci	0512119140
Christian Casale	0512119842

Scritto da:	Christian Casale
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
18/12/2025	1.1	Inizio stesura introduzione e package	Samuele Colucci
18/12/2025	1.2	Stesura e completamento package e class Interface	Samuele Colucci
18/12/2025	1.3	Stesura Componenti off-the-shelf e Design Patterns	Christian Casale
20/12/2025	1.4	Aggiunta diagrammi	Christian Casale
21/12/2025	1.5	Modifiche class interface	Christian Casale

Contents

1.	Introduction	4
1.1	Object design trade-offs	4
1.2	Interface documentation guidelines	7
1.3	Definitions, acronyms, and abbreviations	7
1.4	References	7
2.	Packages	7
2.1	it.unisa.medibook.model	8
2.2	it.unisa.medibook.storage	8
2.3	it.unisa.medibook.business	8
2.4	it.unisa.medibook.presentation	9
3.	Class interfaces	9
3.1	Package it.unisa.medibook.model	Error! Bookmark not defined.
3.2	Package it.unisa.medibook.business	10
3.3	Package it.unisa.medibook.storage	12
4.	Glossary	14

1. Introduction

1.1 Object design trade-offs

Durante l'object design di MediBook, sono stati bilanciati i seguenti criteri contrastanti:

- **Velocità di sviluppo vs. Manutenibilità:** Si è scelto di utilizzare il framework ReactJS per il frontend e Spring-like patterns (DAO/MVC) per il backend. Sebbene richiedano più tempo nella configurazione iniziale, garantiscono una separazione dei compiti che facilita la manutenzione futura (NFR5).
- **Sicurezza vs. Prestazioni:** L'uso di protocolli HTTPS e la crittografia delle password nel database introducono un overhead computazionale, considerato però necessario per rispettare la normativa GDPR (NFR8).
- **Semplicità vs. Flessibilità:** Si è data priorità alla semplicità. L'architettura è stata progettata per rispondere esclusivamente ai requisiti attuali del sistema, evitando l'uso di interfacce eccessivamente generiche o design pattern complessi (come Factory o Strategy spinte) che avrebbero aumentato inutilmente la curva di apprendimento e la complessità del diagramma delle classi. Questo approccio garantisce un codice più snello, facile da testare e immediatamente comprensibile (NFR5).

1.1.1 Componenti off-the-shelf

Per facilitare e velocizzare lo sviluppo della piattaforma MediBook, saranno utilizzate diverse componenti off-the-shelf che riportiamo di seguito.

- **Java Mail**
JavaMail API è un framework che permette la creazione e l'invio di mail attraverso applicazioni scritte in Java ed è incluso sia all'interno di Java SE sia Java EE. Tale libreria verrà utilizzata per l'invio delle mail.
- **MySql-JDBC**
MySQL Connector/J (JDBC) è il driver che permette l'interoperabilità fra Java e MySQL. Viene impiegato per permettere alla WebApp di interfacciarsi col Database, e dunque compiere operazioni CRUD.

- **Spring**

Spring è un framework java pensato per lo sviluppo di applicazioni EE.

1.1.2 Design Patterns

- **Facade**

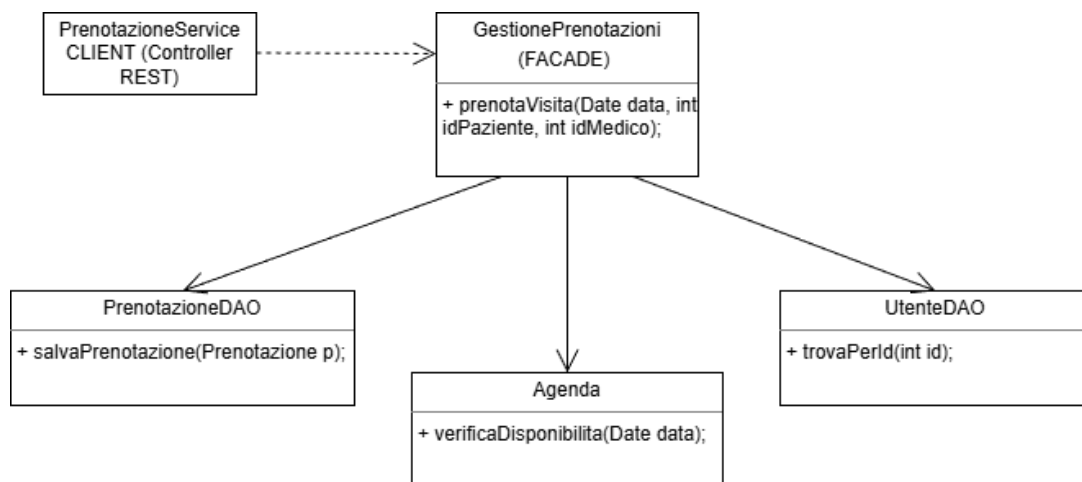
Necessario per nascondere la complessità del backend ai servizi REST. Senza di questo, i tuoi controller dovrebbero chiamare direttamente i DAO, violando la stratificazione.

- **Riferimento teorico:** Il pattern fornisce un'interfaccia unificata a un insieme di interfacce in un sottosistema, definendo un livello più alto che rende il sottosistema più facile da usare.

- **Implementazione in MediBook:**

- **Dove:** Package it.unisa.medibook.business.
- **Perché:** La classe GestionePrenotazioni funge da Facade.
Il metodo prenotaVisita() nasconde la complessità di dover interagire contemporaneamente con PrenotazioneDAO (per salvare), Agenda (per verificare disponibilità) e UtenteDAO (per associare il paziente).
Il Controller REST (PrenotazioneService) chiama solo questo metodo semplice senza conoscere la logica sottostante.

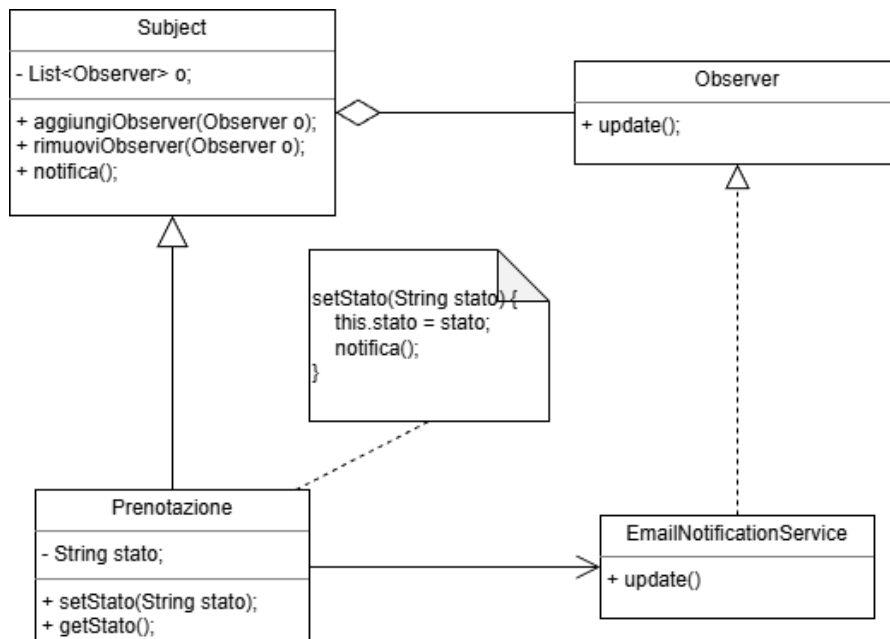
- **Diagramma:**



- **Observer**

Essenziale per gestire le notifiche (email/avvisi) senza mischiare la logica di invio email con quella di salvataggio nel database.

- **Riferimento teorico:** Definisce una dipendenza uno-a-molti tra oggetti in modo che quando uno cambia stato, tutti i dipendenti vengono notificati automaticamente.
- **Implementazione in MediBook:**
 - **Dove:** Tra `it.unisa.medibook.model` (stato prenotazione) e i servizi di notifica.
 - **Perché:** Quando lo stato di una Prenotazione cambia (es. da "Attiva" a "Cancellata"), il sistema deve notificare il paziente. Usando l'Observer, l'oggetto Prenotazione (Subject) notifica semplicemente il cambiamento. Un componente separato (Observer) ascolta questo evento e invia l'email. Questo ti permette di modificare il sistema di notifiche in futuro senza toccare il codice della prenotazione.
- **Diagramma:**



1.2 Interface documentation guidelines

Per garantire la coerenza tra i membri del team durante l'implementazione in Java e ReactJS, sono state stabilite le seguenti convenzioni:

- **Naming Classi:** Utilizzo di sostantivi singolari in CamelCase (es. Prenotazione, Referto).
- **Naming Metodi:** Utilizzo di frasi verbali in lowerCamelCase (es. effettuaPrenotazione(), caricaReferto()).
- **Gestione Errori:** Gli errori (es. "Autenticazione fallita") devono essere gestiti tramite eccezioni personalizzate (AuthenticationException), non tramite valori di ritorno.
- **Interfacce DAO:** Ogni classe del dominio deve avere un'interfaccia DAO corrispondente per l'accesso ai dati persistenti in MySQL.

1.3 Definitions, acronyms, and abbreviations

- ODD: Object Design Document.
- DAO: Data Access Object (Pattern per l'accesso ai dati)
- DTO: Data Transfer Object (Oggetti per il passaggio dati tra i layer)
- API: Application Programming Interface
- MVC: Model-View-Controller

1.4 References

- Bernd Bruegge & Allen H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java, 3rd edition
- Slide del corso di Ingegneria del Software (Prof. De Lucia)
- Documento RAD
- Documento SDD
- Requisiti non funzionali: Sezione 3.3 del RAD

2. Packages

La decomposizione in package del sistema MediBook riflette l'architettura a tre livelli (Three-

	Ingegneria del Software	Pagina 7 di 15
--	-------------------------	----------------

Tier Architecture) definita nel documento di System Design. Il codice sorgente Java è organizzato per separare chiaramente le responsabilità di visualizzazione (gestita via API per il frontend ReactJS), logica di business e persistenza dei dati.

Di seguito viene fornita una descrizione dettagliata dei package identificati nel diagramma soprastante.

2.1 it.unisa.medibook.model

Rappresenta il Model (Entity). Contiene i JavaBeans che mappano le tabelle del database.

Classi Contenute:

Utente: Classe padre per la gestione dei dati di accesso (email, password, ruolo).

Paziente: Estende Utente, include dati anagrafici e codice fiscale.

Medico: Estende Utente, include specializzazione e albo.

Segreteria: Estende Utente, gestisce le operazioni amministrative.

Prenotazione: Rappresenta l'appuntamento (data, ora, stato, riferimenti a Medico e Paziente).

Referto: Contiene i metadati del file PDF caricato e la data di caricamento.

Agenda: Rappresenta la disponibilità oraria del medico.

2.2 it.unisa.medibook.storage

Rappresenta il Model (Persistenza). Contiene le classi DAO per l'accesso al database.

Classi Contenute:

ConnessionePool: Gestisce il pool di connessioni al database MySQL.

UtenteDAO: Metodi CRUD per utenti (doRetrieveByEmail).

PrenotazioneDAO: Gestione delle query per appuntamenti (doRetrieveByMedico, doSave).

RefertoDAO: Gestione del salvataggio dei percorsi dei file referto.

MedicoDAO: Query specifiche per filtrare medici per specializzazione.

2.3 it.unisa.medibook.business

Rappresenta il Controller (Logica). Contiene i manager che coordinano le operazioni.

Classi Contenute:

GestioneUtenza: Gestisce login, logout, e registrazione (login(), registraPaziente()).

GestionePrenotazioni: Valida la disponibilità degli slot e crea le prenotazioni (prenotaVisita(), annullaPrenotazione()).

GestioneReferti: Gestisce l'upload sicuro dei file e l'associazione alla visita (caricaReferto()).

GestioneAgenda: Permette al medico di definire o modificare i propri orari.

2.4 it.unisa.medibook.presentation

Rappresenta la View/Controller (Interfaccia). Poiché l'architettura prevede ReactJS (come definito nell'SDD), in questo package sono elencati i Controller REST (lato Java) e i corrispondenti Componenti React (lato Client) che fungono da vista (al posto delle classiche JSP).

Classi Java (API REST):

AuthService: Endpoint per autenticazione (/auth/login).

PrenotazioneService: Endpoint per gestione visite (/prenotazioni).

MedicoService: Endpoint per ricerca medici e agenda (/medici).

UploadService: Endpoint per il caricamento dei referti (/referti).

Componenti React (View): (Questi file risiedono nel progetto Frontend, ma sono logicamente associati a questo layer)

LoginPage.jsx: Modulo di accesso utente.

RegisterPage.jsx: Form di registrazione paziente.

HomePaziente.jsx: Dashboard paziente con riepilogo.

PrenotaVisita.jsx: Wizard per selezione specialista, data e ora.

GestioneAgenda.jsx: Calendario interattivo per il medico.

DashboardSegreteria.jsx: Tabella gestione appuntamenti odierni.

DettaglioVisita.jsx: Visualizzazione stato visita e download referto.

3. Class interfaces

3.1 Package **it.unisa.medibook.business**

Questo package contiene i Controller. Qui l'enfasi è sulle Pre-condizioni (cosa deve essere vero prima di chiamare il metodo) e Post-condizioni (cosa è garantito dopo).

- **Class GestionePrenotazioni:** Classe Manager responsabile della logica di business per le visite.

Attributes:

dao: PrenotazioneDAO

Operations:

+ prenotaVisita(p: Prenotazione): Boolean

Descrizione: Tenta di registrare una nuova visita.

Constraints (OCL):

context GestionePrenotazioni::prenotaVisita(p: Prenotazione): Boolean

-- Pre: La prenotazione passata non deve essere nulla

pre: p <> null

-- Pre: La data della visita deve essere nel futuro

pre: p.data > Date.now()

-- Pre: Non deve esistere un'altra prenotazione per lo stesso medico alla stessa ora (Slot Libero)

pre: not Prenotazione.allInstances()->exists(existP | existP.medico = p.medico
and existP.data = p.data and existP.ora = p.ora and existP.stato <>
'ANNULLATA')

-- Post: Se il risultato è true, il numero totale di prenotazioni è aumentato di 1

post: result = true implies Prenotazione.allInstances()->size() =

Prenotazione.allInstances@pre()->size() + 1

+ annullaPrenotazione(id: Integer): Void

Descrizione: Annulla una visita esistente se rispetta i tempi limite.

Constraints (OCL):

context GestionePrenotazioni::annullaPrenotazione(id: Integer)

-- Pre: Deve esistere una prenotazione con quell'ID

pre: Prenotazione.allInstances()->exists(p | p.id = id)

-- Pre: La prenotazione identificata deve essere annullabile (regola delle 24h)

pre: let target = Prenotazione.allInstances()->select(p | p.id = id)-
>asSequence()->first() in target.isAnnullabile() = true

```
-- Post: Lo stato della prenotazione diventa 'ANNULLATA'
post: let target = Prenotazione.allInstances()->select(p | p.id = id)-
>asSequence()->first() in target.stato = 'ANNULLATA'
```

Operations:

+ isAnnullabile(): Boolean

```
context Prenotazione::isAnnullabile(): Boolean post: result =
(self.data.difference(Date.now())) > 24.0
-- Ritorna true se mancano più di 24 ore
```

- **Class GestioneUtenza:** Manager per autenticazione.

Operations:

+ login(email: String, pass: String): Utente

Constraints (OCL):

```
context GestioneUtenza::login(email: String, pass: String): Utente
pre: email <> null and pass <> null
-- Post: Se esiste un utente con quella email e password, viene restituito, altrimenti
null
post: result = Utente.allInstances()->select(u | u.email = email and
u.checkPassword(pass))->asSequence()->first()
```

Operazione:

+ registraPaziente(p: Paziente): Boolean

Constraints (OCL):

```
context GestioneUtenza::registraPaziente(p: Paziente): Boolean
-- Pre: L'oggetto paziente non deve essere nullo
pre: p <> null

-- Pre: Non deve esistere già un utente con la stessa email o codice fiscale
pre: not Utente.allInstances()->exists(u | u.email = p.email or
(u.oclIsKindOf(Paziente) and u.oclAsType(Paziente).codiceFiscale =
p.codiceFiscale))

-- Post: Se l'operazione ha successo (true), il numero totale di pazienti è aumentato di
1
post: result = true implies Paziente.allInstances()->size() =
Paziente.allInstances@pre()->size() + 1
```

- **Classe GestioneReferti:** Questa operazione coordina il caricamento di un referto digitale da

parte del medico.

Operazione:

+ caricaReferto(r: Referto, p: Prenotazione): Boolean

Constraints (OCL):

context GestioneReferti::caricaReferto(r: Referto, p: Prenotazione): Boolean

-- Pre: Referto e Prenotazione non nulli

pre: r <> null and p <> null

-- Pre: Si può caricare un referto solo se la visita risulta "EFFETTUATA"

pre: p.stato = 'EFFETTUATA'

-- Post: Il referto r è ora associato correttamente alla prenotazione p

post: result = true implies p.referto = r

3.2 Package it.unisa.medibook.storage

- **Class PrenotazioneDAO:** Classe di accesso ai dati per le prenotazioni.

Operations:

+ doSave(p: Prenotazione): Boolean

Constraints (OCL):

context PrenotazioneDAO::doSave(p: Prenotazione): Boolean

pre: p <> null

pre: p.id = null -- L'ID viene generato dal DB, quindi deve essere nullo prima del salvataggio

post: result = true implies p.id <> null -- Dopo il salvataggio l'oggetto ha un ID assegnato

+ doRetrieveByMedico(idMedico: Integer, data: Date): Set(Prenotazione)

Constraints (OCL):

context PrenotazioneDAO::doRetrieveByMedico(idMedico: Integer, data: Date): Set(Prenotazione)

-- Post: Tutte le prenotazioni restituite appartengono a quel medico e a quella data

post: result->forAll(p | p.medico.id = idMedico and p.data = data)

- **Classe RefertoDAO:** Gestisce la persistenza dei metadati del referto.

Operazione:

+ doSave(r: Referto): Boolean

Constraints (OCL):

context RefertoDAO::doSave(r: Referto): Boolean

-- Pre: Il referto non deve essere nullo

pre: r <> null

-- Pre: L'ID deve essere nullo perché sarà autogenerato dal DB (MySQL)

pre: r.id = null

-- Post: Se salvato correttamente, il referto ha ora un ID assegnato

post: result = true implies r.id <> null

- **Classe MedicoDAO:** Permette la ricerca dei medici filtrando per specializzazione.

Operazione:

+ doRetrieveBySpecializzazione(s: String): Set(Medico)

Constraints (OCL):

context MedicoDAO::doRetrieveBySpecializzazione(s: String): Set(Medico)

-- Pre: La stringa di ricerca non deve essere vuota

pre: s <> "

-- Post: Tutti i medici nel set restituito appartengono alla specializzazione richiesta

post: result->forAll(m | m.specializzazione = s)

- **Class UtenteDAO:** Classe di accesso ai dati per la gestione dell'entità Utente.

Operations:

+ doRetrieveByEmail(email: String): Utente

Constraints (OCL):

context UtenteDAO::doRetrieveByEmail(email: String): Utente

-- Pre: L'indirizzo email fornito non deve essere nullo o una stringa vuota

pre: email <> null and email <> "

-- Post: Se esiste un utente con quell'email, il risultato è l'utente stesso, altrimenti è null

post: result = Utente.allInstances()->select(u | u.email = email)->asSequence()->first()

-- Post: Se il risultato non è nullo, l'email dell'utente restituito deve coincidere con

l'input

post: result <> null implies result.email = email

4. Glossary

Questa sezione fornisce le definizioni dei termini tecnici, degli acronimi e delle tecnologie menzionate all'interno del presente documento di Object Design.

- **API (Application Programming Interface)** Insieme di definizioni e protocolli che permettono al frontend (ReactJS) di comunicare con il backend (Java) per lo scambio di dati.
- **CSS (Cascading Style Sheets)** Linguaggio utilizzato per definire la formattazione e lo stile grafico dei componenti dell'interfaccia utente nel pacchetto presentation.
- **DAO (Data Access Object)** Design pattern strutturale utilizzato nel pacchetto storage per isolare la logica di business dai dettagli di accesso al database. Fornisce un'interfaccia astratta per le operazioni CRUD.
- **DBMS (Database Management System)** Software progettato per definire, manipolare, recuperare e gestire i dati in un database. Nel progetto MediBook viene utilizzato MySQL 8.0.
- **Java** Linguaggio di programmazione orientato agli oggetti, fortemente tipizzato, utilizzato per lo sviluppo del backend e della logica di business del sistema.
- **JavaBeans (Entity)** Classi Java conformi a convenzioni specifiche (costruttore vuoto, attributi privati, metodi getter/setter) utilizzate nel pacchetto model per incapsulare i dati degli oggetti del dominio (es. Prenotazione, Paziente).
- **JDBC (Java Database Connectivity)** API standard di Java che definisce come il client può accedere a un database. È utilizzata all'interno delle classi DAO per eseguire le query SQL.
- **JSON (JavaScript Object Notation)** Formato leggero per lo scambio di dati, facile da leggere e scrivere per gli esseri umani e facile da generare e analizzare per le macchine. È il formato utilizzato per le risposte delle API REST nel pacchetto presentation.
- **MVC (Model-View-Controller)** Pattern architetturale che separa la logica di presentazione (View/React), la logica di business (Controller/Business) e la gestione dei dati (Model/Storage-Entity).
- **OCL (Object Constraint Language)** Linguaggio formale utilizzato nella Sezione 3 di questo documento per descrivere in modo non ambiguo le invarianti di classe, le pre-condizioni e le post-condizioni dei metodi.
- **ReactJS** Libreria JavaScript open-source utilizzata per costruire interfacce utente basate su

componenti. Costituisce la parte "View" dell'architettura.

- Servlet Componente software lato server che gestisce le richieste HTTP e fornisce risposte. Nel pacchetto presentation, le Servlet fungono da entry-point per le chiamate REST.
- SQL (Structured Query Language) Linguaggio standardizzato per la gestione e l'interrogazione di database relazionali.